

Assignment: Apache Spark

ECE 454: Distributed Computing

A few house rules

Collaboration:

- Groups of up to 3 students

Managing source code:

- Please keep a backup copy of your code outside of ECE servers, for example on GitLab (<https://git.uwaterloo.ca/>)
- Please do not post your code in a public repository (e.g., GitHub free tier)

Software environment:

- Please test your implementation on ECE servers
- No other third-party code or external libraries are permitted

Learning objectives

- How to implement a distributed application in Spark.
- How to use a modern Large Language Model (LLM).
- How to implement and evaluate an LLM-based long document summarization solution.

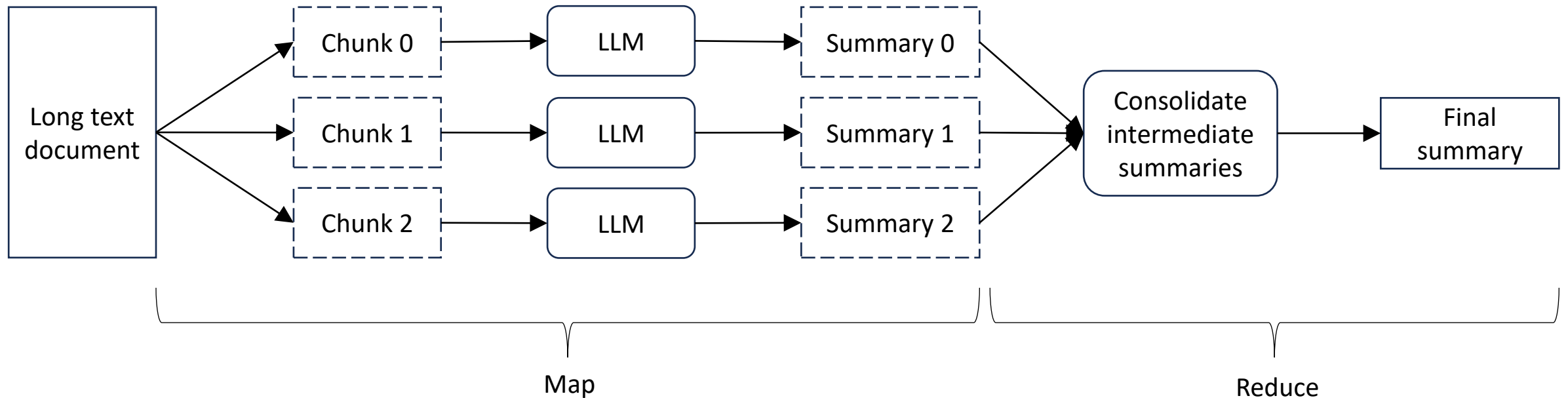
The problem

- Document summarization is one of the most common use cases of LLMs. We are interested in “abstractive summarization”, which generates new sentences that rephrase the original text.
- Summarizing documents of arbitrarily long lengths is challenging for LLMs. Modern LLMs process text token-by-token. A key concept in LLMs is Tokenization - the process of breaking text into smaller units called tokens. The maximum amount of text, that an LLM can reason over, is restricted by the token limit of LLM’s context window.
- Long document summarization can be constrained by a model’s context window as the entire text may not be loaded into LLM for reasoning.
- Also, even for models that offer a large context window, long inputs often lead to degraded reasoning performance, and therefore, are discouraged by domain experts.

Map-Reduce in LLM reasoning

- The so-called Map-Reduce approach in LLM reasoning addresses the aforementioned limitation of LLMs when working with long documents. The technique is composed of two key steps:
 - Map: a document is split into token-bounded chunks so that each chunk can fit within the LLM's context window (or some token limit). The LLM produces an intermediate summary for each chunk. Depending on available resources, chunks can be processed in parallel (on multiple GPUs) or iteratively (on a single GPU).
 - Reduce: the per chunk, intermediate summaries generated in the map phase are combined into a single consolidated summary.
 - Note that, until the combined summary is reduced to a predefined output size (often measured in number of tokens) or fit the LLM's context window, the map-reduce phases are iterated.
- Well-known frameworks like LangChain uses map-reduce for long document reasoning.

Map-Reduce in LLM reasoning



A high-level illustration of a distributed LLM-based long text summarization workflow
(here, high parallelism can be achieved by processing chunks in parallel on a distributed platform)

Tokenization

- A key concept in LLMs is Tokenization - the process of breaking text into smaller units called tokens that an LLM can process. The maximum amount of text, that an LLM can reason over, is restricted by token limits of its context window.
- Each token is assigned a unique numeric ID. Each ID is transformed into a token embedding (vector) that LLMs operate on for reasoning.
- Modern LLMs commonly use subword tokenization. Byte-Pair Encoding (BPE) is a widely used subword tokenization technique.

Tokenization

- For example, BPE tokenization of the following sentence (generated using the tiktoken library used in GPT models):
 - The quick brown fox jumped over the lazy dog.
 - ['The', ' quick', ' brown', ' fox', ' jump', 'ed', ' over', ' the', ' lazy', ' dog', '.']
- The token IDs of input text can be obtained from the vocabulary of the LLM of interest. For example, GPT-4 uses a vocabulary of 100,256 tokens.

Evaluating quality of generated summary

- We will use the popular [ROUGE](#) (Recall-Oriented Understudy for Gisting Evaluation) metric for evaluating quality of generated summary.
- We will compare the summary produced by your implementation (called the candidate summary) against an LLM-generated reference summary.

$$\text{ROUGE score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

- At the high-level, ROUGE measures the overlap of individual words or contiguous sequence of words between the candidate and reference summaries. ROUGE scores range from 0.0 to 1.0, a score of 1.0 indicates an exact match.

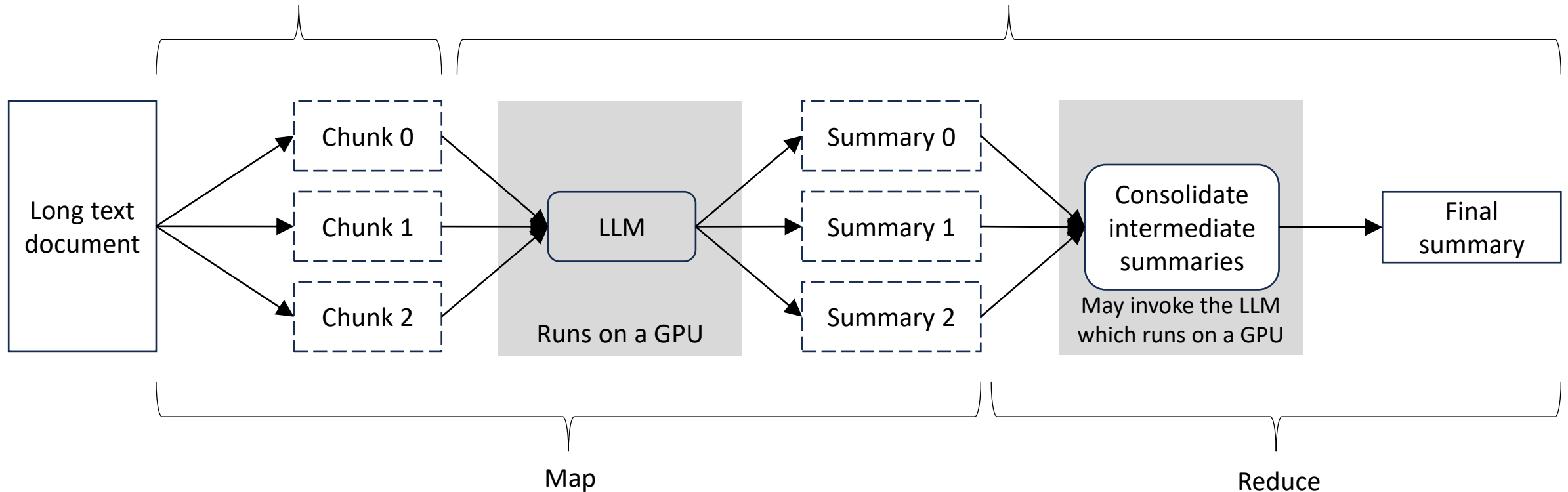
Objective

- In this assignment, you will work with an LLM-based distributed document summarization solution. For implementation, you will use PySpark, the Python API for Apache Spark.
- The document summarization solution follows the map-reduce approach discussed earlier.
- Your task is to implement part of the map phase in PySpark.
- You will run the application on a single host with a single GPU.
- A high-level illustration of the document summarization workflow is presented in the next slide. This setup looks a little different than the one [here](#) because we are limited by a single host, single GPU.

Objective

You will implement part of the map phase in Spark

Implementation provided with the starter code



The LLM runs on a single GPU, processes the chunks iteratively, one at a time.
Only the part with a **solid gray background** is run on the GPU. The rest of the workflow is run on the CPU.

Dataset

- The dataset used in this assignment is adapted from the ArXiv Summarization* dataset available on [Hugging Face](#). This is a collection of scientific papers obtained from ArXiv and PubMed OpenAccess repositories (over 400K papers). This dataset is widely used for evaluating neural abstractive summarization models.
- Two input sets are provided for your use: (i) small.csv (16 articles) - for your use during development, and (ii) large.csv (6,440 articles) - for performance evaluation. A dataset of size comparable to large.csv will be used for evaluating performance during assignment grading.
- A reference summary of each paper, used as the ground truth, is provided.

* Originally by Cohan et al. <https://github.com/armancohan/long-summarization>

Objective

- The goal is, using Spark distributed computing, to transform a large document (collection of papers) into an input format that can be used by an LLM to generate document summary (of each paper). (Part of the map phase [here](#)).

Input for the map phase (we will use two input sets, small.csv and large.csv):

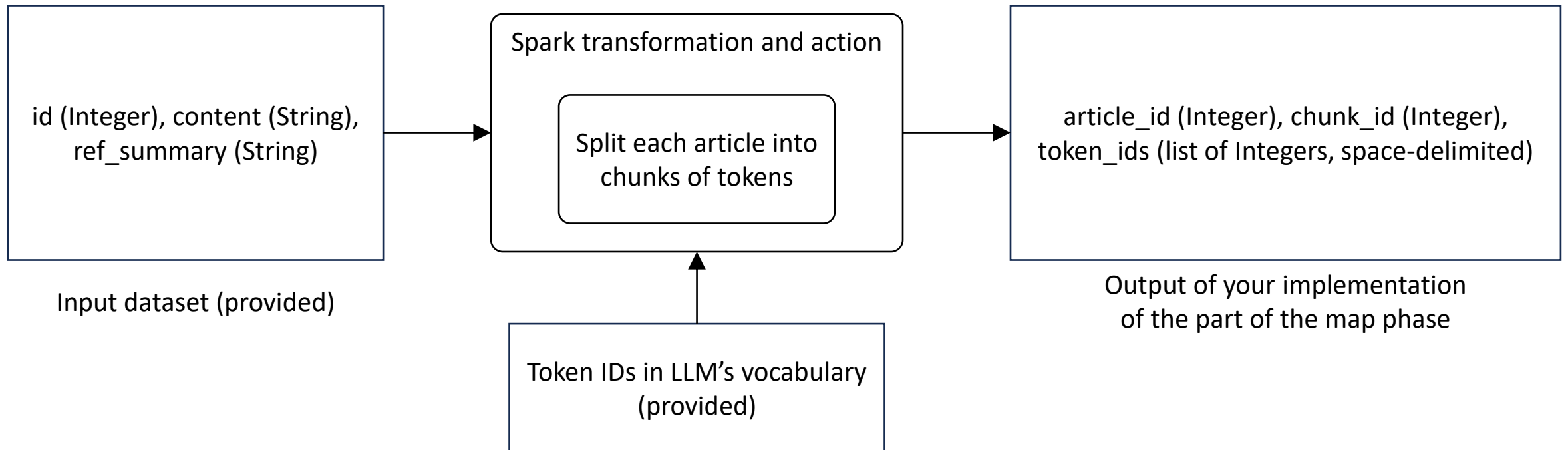
- The input is a collection of papers in the csv format. The column headers are:
 - id (Integer), content (String), ref_summary (String)
- The “content” column contains the papers. The “ref_summary” column contains LLM generated summary of the corresponding paper, uniquely identified by the number under the “id” column.

Objective

Output of the part of the map phase that you will implement in Spark:

- The column headers of the output file are:
 - `article_id` (Integer), `chunk_id` (Integer), `token_ids` (list of space-delimited Integers, each representing a token ID)
- The “`article_id`” column contains the article/paper IDs (as in the input articles).
- An article must be split into chunks where the number of tokens in a chunk is bounded by the predefined maximum chunk size (512 tokens).
- The “`chunk_id`” column contains zero-indexed chunk IDs of the article identified in the “`article_id`” column of the same row.
- The “`token_ids`” column contains the IDs of tokens (space-delimited) in the corresponding chunk of an article.

Objective



A more focused depiction of part of the map phase that you will implement in Spark

Objective

Splitting each article into chunks of tokens

- In our problem setup, the context window size (number of input tokens) of the LLM is restricted to 512 tokens. In subword tokenization, 512 tokens equals to about 350 words and a typical scientific paper has at least 4000 words. Therefore, each article must be split into multiple smaller texts. Each smaller text must be tokenized, and tokens for each smaller text (i.e., a chunk) are presented to the LLM for summarization.
 - An implementation of reduce, i.e., the process of creating the final summary (limited to 100 tokens) of an article from intermediate summaries generated in the map phase, is provided with the starter code.
- The token IDs for the input text can be obtained from the vocabulary of the LLM being used (token IDs are provided with the starter code).

Working with the starter code package

- The starter code “STARTER_a4.tar.gz” tarball is available on LEARN.
- The “a4_starter” directory contains a partial implementation and required execution scripts.
- Your task is to implement part of the Map phase in Spark.
- Please add your implementation to “generate_chunks.py”.
- Please do not modify other files in a4_starter.
- Please update permissions of the scripts by running “chmod +x <script_name>.sh”.
- Output files from the last run are overwritten every time the starter code is run.

Working with the starter code package

Computing environment

- Five servers, each equipped with an AMD Radeon RX 7900 XTX (24GB), are available for this assignment. The machines are to be shared by about 35 groups. Hostname of machines:
 1. ece-7900xtx-09.eng.uwaterloo.ca
 2. ece-7900xtx-10.eng.uwaterloo.ca
 3. ece-7900xtx-11.eng.uwaterloo.ca
 4. ece-7900xtx-12.eng.uwaterloo.ca
 5. ece-7900xtx-13.eng.uwaterloo.ca
- The map-reduce application is run on a single machine; no inter-node communication is incurred.
- LLM and input datasets are locally stored on each machine. LLM and input dataset files paths are included in the starter code.
- A shared, read-only virtual environut with required software/packages has been created on each machine.
- **Before you run the starter code, please activate the environment** required for this assignment:
 - `source /opt/shared_envs/a4_venv/bin/activate`

Working with the starter code package

- Please implement your Spark code in “generate_chunks.py”. The file contains instructions about where to add your code.
- You need to implement how to transform each article in the input document into chunks of token IDs in the required [output format](#).
- The provided “encode_sentence()” function in generate_chunks.py turns a given text into token IDs using a BPE subword tokenizer. The starter code includes an example.
- You are free to base your solution on Spark RDDs or DataFrames, or a combination of both.

Notes on splitting articles into chunks of tokens

- In the second part of the map phase, the LLM generates an intermediate summary for each token chunk. In the reduce phase, for each article, a single summary is constructed from its intermediate summaries.
- For effectiveness of summary generation, we should present complete sentences to the LLM. Therefore, it is preferred that the text splitting process is sentence-aware, i.e., we want to avoid cutting off sentences and a chunk should include tokens from one or more complete sentences.
- An artifact of creating sentence-aware chunks is that the chunk sizes may vary significantly, potentially creating more chunks compared to a sentence-oblivious text splitting approach, and increasing summary generation latency in the process.

Working with the starter code package

The starter code is divided into a number of steps that can be run independently:

- i. Generating token chunks/input for LLM summarization (this is part 1 of the map phase; we measure runtime of this step)
- ii. LLM summarization using token chunks created in step i (part 2 of the map phase and the reduce phase)
- iii. Computing ROUGE-1 scores of generated summaries in step ii.
- iv. Inspecting the chunks identified in the map phase (optional)

Generating token chunks/input for LLM summarization

- “run_generate_chunks.sh” executes “generate_chunks.py” which contains a partial implementation or your complete implementation of part of the map phase. Example:
 - ./run_generate_chunks.sh small
 - ./run_generate_chunks.sh large # please run on the large dataset only when you have reasonable confidence in your implementation
- The output is written in /result/chunks_<dataset>/. (This is the input for the next part of the map phase.)
 - Colum headers: article_id, chunk_id, token_ids
- Runtime of the Spark code is output in result/performance_<dataset>.csv.
 - Colum header: elapsed_seconds
- run_generate_chunks.sh will automatically terminate after 5 minutes.

LLM summarization using token chunks

- “run_generate_summaries.sh” invokes “generate_summaries.py” which generates article summaries using the provided LLM.
 - ./run_generate_summaries.sh small # **please do not run this on the large dataset**
- /result/chunks_<dataset>/, the output of the previous step, is the input for generate_summaries.py.
- Token chunks in /result/chunks_<dataset>/ are iteratively processed on a single GPU.
- Please note that this step is expensive. For the small dataset, the reference implementation took over 5 minutes.
- **Please DO NOT run this on the token chunks of the large dataset. In our evaluation, it did not finish within 24 hours.**
- run_generate_summaries.sh will automatically terminate after 15 minutes.

Computing ROUGE-1 scores of generated summaries

- The “run_evaluate_summaries.sh” script computes ROUGE-1 scores of generated summaries by comparing them against reference summaries generated using LLM.
 - Example: `./run_evaluate_summaries.sh small`
- The output is written in `result/score_<dataset>.csv`.
 - Column headers: `id, rouge1`

Inspecting the chunks identified in the map phase

After “run_generate_chunks.sh” has been completed, you may use “run_inspect_chunks.sh” for verifying the output. Example:

- `./run_inspect_chunks.sh small #` for all articles in the dataset; please do not run this command for the large dataset
- `./run_inspect_chunks.sh small --article-id 81 #` for a single article in the dataset
- `./run_inspect_chunks.sh small --stats-only #` outputs statistic

Working with the starter code package

Notes on dataset usage

Map phase (Spark executor uses up to 8 threads on the CPU):

`./run_generate_chunks.sh small # please use the small dataset during development`

`./run_generate_chunks.sh large # please run on the large dataset only when you have reasonable confidence in your implementation; inefficient Spark code can be very slow, may get stuck`

Reduce phase (runs LLM inference on the GPU):

`./run_generate_summaries.sh small # please do not run this on the large dataset; run on the small dataset only when you have reasonable confidence in your implementation`

Evaluate quality of generated summaries:

`./run_evaluate_summaries.sh small # to compute ROUGE-1 scores of generated summaries, please run on the small dataset only`

Working with the starter code package

- LLM (included with the starter code): [HuggingFaceTB/SmolLM3-3B](https://huggingface.co/HuggingFaceTB/SmolLM3-3B)
 - Model size: 3B parameters
- Token limit for a chunk of an article: 512
- Token limit for an intermediate/the final summary of an article: 100 (you do not need to worry about this)
- Input (included in the scripts):
 - /local/a4/inputs/small.csv
 - /local/a4/inputs/large.csv
- LLM vocabulary (included in the scripts):
 - /local/tokenizer_files/vocab.json
 - /local/tokenizer_files/merges.txt
- Output
 - /a4_starter/result/chunks_<dataset>/
 - /a4_starter/result/performance_<dataset>.csv
 - /a4_starter/result/scores_<dataset>.csv
 - /a4_starter/result/summaries_<dataset>.csv

Server usage

- Since each server has only one GPU (AMD Radeon RX 7900 XTX 24GB), `run_generate_summaries.sh`, which invokes the LLM on the GPU, will only execute if the GPU is not being used.
- If the GPU is busy, `run_generate_summaries.sh` will wait for a very short time before exiting. Simply try later; if the GPU is free, `run_generate_summaries.sh` will complete successfully.
- Also, “amd-smi monitor” displays GPU activity.
- Please do not wait until near the due date to start working on the assignment. If many users are trying to run their code at the same time on the same server, the wait time will likely be very high.

Ideas for improving performance

- If `run_generate_chunks.sh` is running slow or reaching the time limit before producing the target output, especially for the large dataset, please focus on optimizing your Spark code, e.g., avoiding redundant work, minimizing the number of job stages, the number of data shuffles, and the amount of time spent in garbage collection to name a few. (Part of assignment evaluation.)
- `run_generate_summaries.sh` iteratively processes chunks in an article on the GPU. Minimizing the total number of chunks per article should improve the time required to complete this step. (Not part of assignment evaluation.)

Evaluation

Grading scheme:

Performance: 70%

Result quality: 30%

A penalty of up to 100% will be applied in the following cases:

- Solution cannot be compiled or throws an exception during testing despite receiving valid input
- Solution produces incorrect outputs, for example due to a logic error or concurrency bug

Evaluation

Your implementation will be evaluated on performance of the Spark code and the quality of generated summaries.

- Performance: time (measured in seconds) spent in producing the input required for LLM reasoning in the map phase. File loading and output times are included in this measurement. A dataset of size comparable to large.csv will be used for evaluating performance.
- Quality of generated summary: the ROUGE-1 metric will be used for measuring quality of generated summary. We will compare the summaries produced by your implementation against LLM-generated reference summaries. At most 100 scientific papers will be used for evaluation of quality of generated summary. The average ROUGE-1 score of these papers indicates the overall quality of your implementation.

Packaging and submission

- Please use the provided `package.sh` script to create your tarball for electronic submission, and upload it to the appropriate LEARN dropbox before the deadline.
- All group members must join an A4 group before submitting.

Sample performance numbers and ROUGE scores

- Time spent in producing the input required for LLM reasoning in the map phase.
 - Using spark-submit --master local[8], i.e., 8 worker threads
- small.csv: about 20 seconds
- large.csv: about 2 minutes 30 seconds

Small dataset	
id	rouge1
1264	0.4266666667
1836	0.490797546
1840	0.4946236559
2665	0.7816091954
2681	0.5641025641
2799	0.4482758621
3131	0.7317073171
3211	0.6040268456
4468	0.6117647059
477	0.421875
5063	0.4378698225
5413	0.462585034
5647	0.5088757396
5721	0.5974025974
6327	0.4204545455
81	0.5165562914

Sample output of the Spark code

article_id	chunk_id	token_ids				
1264	0	279 33472 315 279 3177 5540 908 1174 387 1174 323 293 320 3127 3141 883 1782				
1264	1	311 3493 279 5995 828 1174 584 17303 20932 43785 315 571 87 10590 806 90574				
1264	2	264 2218 36865 574 7559 520 279 4219 315 279 2543 2048 1174 323 5762 14580				
1264	3	304 56925 220 16 279 4831 5469 1047 26839 288 315 220 9588 571 87 10590 1114				
1264	4	279 1176 12304 1614 304 571 87 10590 15 747 320 220 17 13 972 105340 883 165				
1264	5	279 828 315 571 8311 635 617 2731 20932 11175 1109 279 3118 9526 1174 323 10				
1264	6	69766 574 4856 3428 1606 279 3428 4907 90574 3742 10717 304 279 220 3101 57				
1264	7	584 9749 430 279 571 87 10590 15 383 5425 3857 520 220 17416 105340 374 275				
1264	8	220 21 527 37924 59855 18809 1555 682 3585 6982 1174 323 617 279 16003 7739				
1264	9	304 5157 1405 571 87 10590 15 383 22323 527 539 2561 1174 279 2694 5764 459				
1264	10	1606 315 279 9333 5425 14491 12457 1618 369 3148 12 21 279 279 5788 315 571				
1264	11	369 279 55801 18803 20326 584 617 12146 1174 279 6811 1990 279 1403 32946 2				
1264	12	342 13 938 437 1174 264 13 308 11354 268 1174 323 10167 296 13 264 45335 1174				
1836	0	279 7479 5415 315 2380 482 56987 320 220 18 67 883 1948 5848 1672 42391 320				
1836	1	1455 21896 1005 264 18728 442 39100 3729 439 264 12903 32314 323 6767 12903				
1836	2	14799 1174 584 690 3118 264 18300 3477 315 279 22027 4920 279 12903 482 182				
1836	3	304 279 1176 5684 555 908 721 571 8311 635 1174 433 574 18784 430 721 8389 7				
1836	4	994 264 2536 482 56411 12903 83245 571 87 10590 605 374 36572 304 279 1554				
1836	5	994 279 259 784 24266 264 6900 1510 571 87 10590 2491 3235 279 571 87 10590				

Please refer to the [output format](#).

Notes on assignment grading

- An ece-7900ctx-##.eng.uwaterloo.ca server will be used for grading students' submissions.
- Your implementation will be run in a single-node, single GPU environment.
- Performance will be graded based on class median runtime of the Spark part of the map phase.
- Result quality will be graded based on class median ROUGE-1 score.